

AI Assisted Coding

Assignment-16

Name: G.Abhilash

HT NO: 2303A52195

Batch: 35

Lab 16 – Database Design and Queries (AI Assisted)

Lab Objectives

- Use AI to assist in designing and implementing fundamental data structures in Python
- Learn prompt-based structure creation, optimization, and documentation
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Graphs, and Hash Tables
- Enhance readability and performance with AI-generated suggestions

Task 1 – Student Information System Schema

1. AI Prompt Used

Generate SQL schema for a Student Information System.

Tables required:

Students

Courses

Enrollments

Requirements:

- Define primary keys
- Define foreign key relationships
- Maintain referential integrity
- Generate SQL queries for:
 - Insert a new student
 - Fetch all courses enrolled by a student
 - Count students in each course

CODE and OUTPUT:

```
Wool
INSERT INTO Courses (course_id, course_name)
VALUES (101, 'Mathematics');

* sqlite:///lab36.db
1 rows affected.
[]

Wool
INSERT INTO Enrollments (student_id, course_id)
VALUES (1, 101);

* sqlite:///lab36.db
1 rows affected.
[]

Wool
CREATE TABLE Students (
  student_id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT UNIQUE
);

CREATE TABLE Courses (
  course_id INTEGER PRIMARY KEY,
  course_name TEXT NOT NULL
);

CREATE TABLE Enrollments (
  enrollment_id INTEGER PRIMARY KEY,
  student_id INTEGER,
  course_id INTEGER,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

-- * sqlite:///lab36.db
Done.
Done.
Done.
[]

Wool
INSERT INTO Students (student_id, name, email)
VALUES (1, 'Akshay', 'akshay@email.com');

* sqlite:///lab36.db
1 rows affected.
[]

import pandas as pd
df = Wool --dataframe SELECT Students.name, Courses.course_name FROM Enrollments JOIN Students ON Students.student_id = Enrollments.student_id JOIN Courses ON Courses.course_id = Enrollments.course_id WHERE Students.student_id = 1;
display(df)

* sqlite:///lab36.db
None

Wool
import pandas as pd
df = Wool --dataframe SELECT Courses.course_name, COUNT(Enrollments.student_id) AS total_students FROM Enrollments JOIN Courses ON Courses.course_id = Enrollments.course_id GROUP BY Courses.course_name;
display(df)

-- * sqlite:///lab36.db
None
```

Students

student_id	name	email
1	Akshay	akshay@email.com
2	Ravi	ravi@email.com

Courses

course_id	course_name
101	Database Systems
102	Machine Learning

Enrollments

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	101

Task 2 – Hospital Management Database AI Prompt

Generate SQL schema for a Hospital Management System.

Tables:

Doctors

Patients

Appointments

Requirements:

- Primary keys
- Foreign keys
- Date validation
- Queries for:
 - List appointments for a doctor
 - Retrieve patient history
 - Count patients treated by each doctor

Schema AND Output:

```
0 CREATE TABLE Patients (
  patient_id INT(10) PRIMARY KEY,
  name VARCHAR(100),
  age INT(3)
);

CREATE TABLE Appointments (
  appointment_id INT(10) PRIMARY KEY,
  doctor_id INT(10),
  patient_id INT(10),
  appointment_date DATE,
  FOREIGN KEY (doctor_id) REFERENCES Doctors (doctor_id),
  FOREIGN KEY (patient_id) REFERENCES Patients (patient_id)
);

-- * sqlite:///hospital.db
Name:
Type:
Data:
[]

import pandas as pd
df = pd.read_sql_query('SELECT Doctors.name, Patients.name, appointment_date FROM Appointments JOIN Doctors ON Doctors.doctor_id = Appointments.doctor_id JOIN Patients ON Patients.patient_id = Appointments.patient_id WHERE Doctors.doctor_id = 1;',
display(df)

-- * sqlite:///hospital.db
Name:

import pandas as pd
df = pd.read_sql_query('SELECT Doctors.name, Patients.name, appointment_date FROM Appointments JOIN Doctors ON Doctors.doctor_id = Appointments.doctor_id JOIN Patients ON Patients.patient_id = Appointments.patient_id WHERE Doctors.doctor_id = 1;',
display(df)

-- * sqlite:///hospital.db
Name:

import pandas as pd
df = pd.read_sql_query('SELECT Doctors.name, COUNT(Patients.patient_id) AS total_patients FROM Appointments JOIN Doctors ON Doctors.doctor_id = Appointments.doctor_id JOIN Patients ON Patients.patient_id = Appointments.patient_id GROUP BY Doctors.name;',
display(df)

-- * sqlite:///hospital.db
Name:

import pandas as pd
df_doctors = pd.read_sql_query('SELECT * FROM Doctors;',
display(df_doctors)

-- * sqlite:///hospital.db
Name:
```

Sample Data

Doctors

doctor_id	name	specialization
1	Dr. Sharma	Cardiology
2	Dr. Rao	Neurology

Patients

patient_id	name	age
1	Arjun	30
2	Meera	25

Appointments

appointment_id	doctor_id	patient_id	appointment_date
1	1	1	2026-03-10
2	1	2	2026-03-11

Task 3 – Library Database

AI Prompt

Generate SQL schema for a Library Management System.

Tables:

Books

Members

Loans

Requirements:

- Add indexing strategy
- Generate queries:
 - Books currently issued
 - Overdue books (loan > 30 days)
 - Books loaned per member

CODE and OUTPUT:

```
CREATE TABLE Books (
  book_id INT PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  author VARCHAR(100) NOT NULL
);

CREATE TABLE Members (
  member_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);

CREATE TABLE Loans (
  loan_id INT PRIMARY KEY,
  book_id INT NOT NULL,
  member_id INT NOT NULL,
  loan_date DATE NOT NULL,
  FOREIGN KEY (book_id) REFERENCES Books (book_id),
  FOREIGN KEY (member_id) REFERENCES Members (member_id)
);

INSERT INTO Books (book_id, title, author) VALUES
(1, 'AI Basics', 'John'),
(2, 'SQL Mastery', 'David');

INSERT INTO Members (member_id, name) VALUES
(1, 'Akshay'),
(2, 'Ravi');

INSERT INTO Loans (loan_id, book_id, member_id, loan_date) VALUES
(1, 1, 1, '2026-02-01');
```

Books			
book_id	title	author	
1	AI Basics	John	
2	SQL Mastery	David	

Members	
member_id	name
1	Akshay
2	Ravi

Loans			
loan_id	book_id	member_id	loan_date
1	1	1	2026-02-01

Task 4 – Online Shopping Database

AI Prompt

Generate SQL schema for an E-commerce platform.

Tables:

Users

Products

Orders

OrderDetails

Queries:

Retrieve all orders by a user

Find most purchased product

Calculate monthly revenue

Also suggest normalization improvements.



CODE and OUTPUT:

```

--Step1
CREATE TABLE Users (
  user_id INT(10) PRIMARY KEY,
  name TEXT,
  email TEXT UNIQUE
);

CREATE TABLE Products (
  product_id INT(10) PRIMARY KEY,
  name TEXT,
  price DEC
);

CREATE TABLE Orders (
  order_id INT(10) PRIMARY KEY,
  user_id INT(10),
  order_date DATE,
  FOREIGN KEY(user_id) REFERENCES Users(user_id)
);

CREATE TABLE OrderDetails (
  order_detail_id INT(10) PRIMARY KEY,
  order_id INT(10),
  product_id INT(10),
  quantity INT(10),
  FOREIGN KEY(order_id) REFERENCES Orders(order_id),
  FOREIGN KEY(product_id) REFERENCES Products(product_id)
);

-- * sqlfiddle:///fiddle.db
Name:
Schema:
Name:
Name:
[]

--
Import pandas on pd
df = pd.read_csv('data/orders.csv')
display(df)

-- * sqlfiddle:///fiddle.db
Name:

Import pandas on pd
df = pd.read_csv('data/products.csv')
display(df)

-- * sqlfiddle:///fiddle.db
Name:

-- * sqlfiddle:///fiddle.db
Name:

--
Import pandas on pd
df = pd.read_csv('data/orders.csv')
display(df)

-- * sqlfiddle:///fiddle.db
Name:

```

Users			
user_id	name		
1	Akshay		
Products			
product_id	name	price	
1	Laptop	60000	
2	Mouse	500	
Orders			
order_id	user_id	order_date	
1	1	2026-03-01	
OrderDetails			
order_detail_id	order_id	product_id	quantity
1	1	1	1
2	1	2	2

Task 5 – University Examination Database
AI Prompt:

Generate SQL schema for a University Examination System.

Tables:

Students

Subjects

Exams

Results

Queries:

Insert exam results

Retrieve subject-wise marks

Calculate average marks per subject

CODE and OUTPUT:

```
SQL
CREATE TABLE Subjects (
  subject_id NUMBER(10) PRIMARY KEY,
  subject_name VARCHAR(100)
);

CREATE TABLE Exams (
  exam_id NUMBER(10) PRIMARY KEY,
  subject_id NUMBER(10),
  exam_date DATE,
  FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);

CREATE TABLE Results (
  result_id NUMBER(10) PRIMARY KEY,
  student_id NUMBER(10),
  exam_id NUMBER(10),
  score NUMBER(5),
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);

-- Sample Data Insertion
INSERT INTO Subjects (subject_id, subject_name) VALUES (1, 'Mathematics');
INSERT INTO Subjects (subject_id, subject_name) VALUES (2, 'Science');
INSERT INTO Subjects (subject_id, subject_name) VALUES (3, 'History');
INSERT INTO Subjects (subject_id, subject_name) VALUES (4, 'Literature');
INSERT INTO Subjects (subject_id, subject_name) VALUES (5, 'Physical Education');

INSERT INTO Exams (exam_id, subject_id, exam_date) VALUES (1, 1, TO_DATE('2023-09-01', 'YYYY-MM-DD'));
INSERT INTO Exams (exam_id, subject_id, exam_date) VALUES (2, 2, TO_DATE('2023-09-01', 'YYYY-MM-DD'));
INSERT INTO Exams (exam_id, subject_id, exam_date) VALUES (3, 3, TO_DATE('2023-09-01', 'YYYY-MM-DD'));
INSERT INTO Exams (exam_id, subject_id, exam_date) VALUES (4, 4, TO_DATE('2023-09-01', 'YYYY-MM-DD'));
INSERT INTO Exams (exam_id, subject_id, exam_date) VALUES (5, 5, TO_DATE('2023-09-01', 'YYYY-MM-DD'));

INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (1, 101, 1, 85);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (2, 102, 1, 78);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (3, 103, 1, 92);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (4, 104, 1, 65);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (5, 105, 1, 88);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (6, 101, 2, 72);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (7, 102, 2, 89);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (8, 103, 2, 75);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (9, 104, 2, 81);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (10, 105, 2, 77);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (11, 101, 3, 83);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (12, 102, 3, 79);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (13, 103, 3, 86);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (14, 104, 3, 74);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (15, 105, 3, 80);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (16, 101, 4, 76);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (17, 102, 4, 82);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (18, 103, 4, 71);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (19, 104, 4, 87);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (20, 105, 4, 73);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (21, 101, 5, 84);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (22, 102, 5, 77);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (23, 103, 5, 89);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (24, 104, 5, 75);
INSERT INTO Results (result_id, student_id, exam_id, score) VALUES (25, 105, 5, 81);

-- Query 1: Retrieve subject-wise marks
SELECT subject_id, subject_name, result_id, student_id, exam_id, score
FROM Results
ORDER BY subject_id, result_id;

-- Query 2: Calculate average marks per subject
SELECT subject_id, subject_name, AVG(score) AS average_marks
FROM Results
GROUP BY subject_id, subject_name;
```


Subjects			
subject_id		subject_name	
1		Database	
2		AI	
Results			
result_id	student_id	exam_id	marks
1	1	1	85
2	1	2	90
Query – Subject Wise Marks			
Output			
student	subject	marks	
Akshay	Database	85	
Akshay	AI	90	

Final Analysis

The experiment demonstrated how AI can assist in designing relational database schemas and generating SQL queries. Multiple database systems were modeled including Student Information Systems, Hospital Management Systems, Library Systems, E-commerce platforms, and University Examination Systems. SQL queries demonstrated CRUD operations, joins, and aggregate functions. Normalization principles and indexing strategies were applied to improve data integrity and query performance.